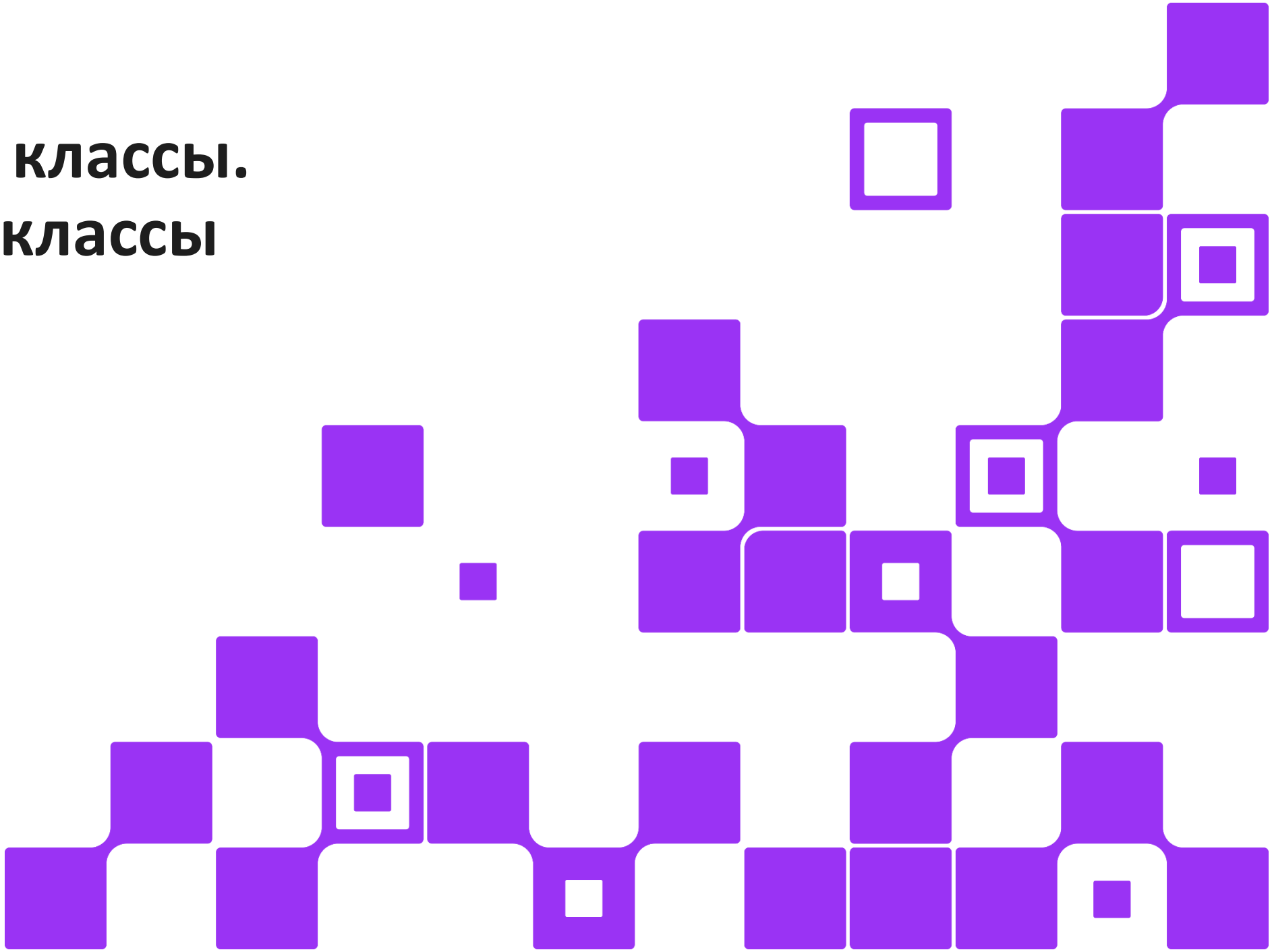
Статические классы.
Вложенные классы



IThub

IThub

Статические классы в Java

Статический класс – это класс, который не может быть инстанцирован, то есть создан как объект, и доступен только через его статические методы и свойства. Статические классы могут содержать только статические методы и свойства, которые могут быть вызваны без создания экземпляра класса.

Одним из наиболее распространенных применений статических классов является группировка связанных методов и свойств в одном месте. Например, статический класс Math в Java содержит методы, которые необходимы для выполнения математических операций, таких как вычисление квадратного корня, синуса и т.д.

IThub

```java
public class UseStatic {

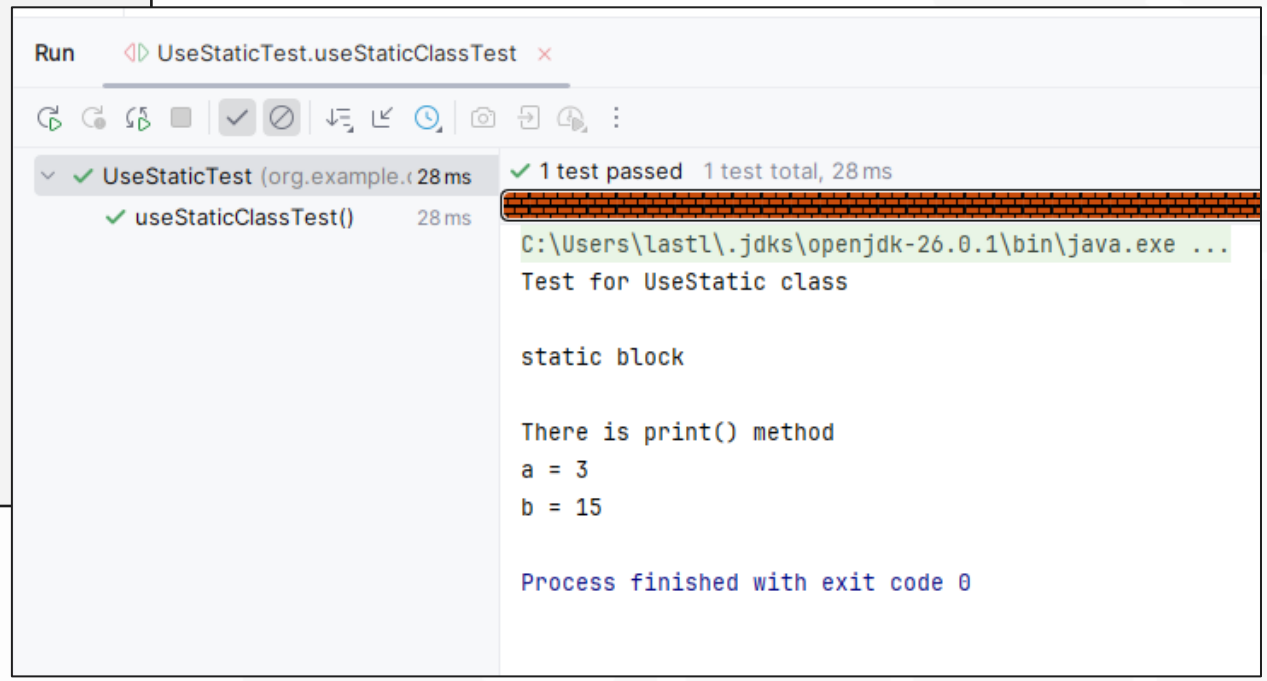
    static int a = 3;
    static int b;

    static void print() {
        System.out.println("\nThere is print() method");
        System.out.println("a = " + a);
        System.out.println("b = " + b);
    }

    static {
        System.out.println("\nstatic block");
        b = a * 5;
    }
}
```

```java
@Test
public void useStaticClassTest() {
    System.out.println("Test for UseStatic class");
    UseStatic.print();

    Assertions.assertEquals(3, UseStatic.a);
    Assertions.assertEquals(15, UseStatic.b);
}
```



Вложенные классы в Java

- класс можно определять внутри другого класса
- внутренний класс имеет доступ ко всем членам окружающего его класса, но обратное неверно

В Java есть четыре типа вложенных классов:

1. статические
2. нестатические (внутренние)
3. Локальные
4. анонимные классы



Статические вложенные классы

Статический вложенный класс – это вложенный класс, который является статическим членом своего внешнего класса. Статические вложенные классы могут быть созданы без создания экземпляра внешнего класса, и **имеют доступ только к статическим** членам внешнего класса.

```java
public class Computer {
    private String model;

    public Computer(String model) { this.model = model; }

    public void showInfo() {
        System.out.println("Компьютер: " + model);
    }

    // Статический вложенный класс
    static class Processor {

        private String name;
        private int cores;

        // конструктор с двумя полями name и cores

        public void showProcessorInfo() {
            System.out.println(
                    "Процессор: " + name +  ", ядер: " + cores  );
        }
    }
}
```

```java
public static void main(String[] args) {

        // Создаём объект внешнего класса
        Computer computer = new Computer("ASUS");

        computer.showInfo();


        // Создаём объект статического вложенного класса
        Computer.Processor processor =
                new Computer.Processor("Intel Core i7", 8);

        processor.showProcessorInfo();
}
```

Нестатические вложенные классы

Нестатический вложенный класс, также называемый **внутренним классом,** – это класс, который связан с экземпляром внешнего класса. Нестатический вложенный класс может иметь доступ к нестатическим членам внешнего класса, включая приватные члены, и может использоваться для реализации более сложных алгоритмов.

```
public class Computer {

    private String model;
    private int batteryLevel;

    // конструктор - public Computer(String model, int batteryLevel)

    public void showInfo() {  System.out.println("Компьютер: " + model);  }

    // нестатический вложенный класс
    class Battery {

        private int capacity;

        // конструктор - public Battery(int capacity)

        public void showBatteryInfo() {
            System.out.println("Модель компьютера: " + model);
            System.out.println("Заряд батареи: " + batteryLevel + "%");
            System.out.println("Ёмкость батареи: " + capacity + " мАч");
        }
    }
}
```

```
public static void main(String[] args) {

        Computer computer = new Computer("Lenovo", 80);

        Computer.Battery battery = computer.new Battery(5000);

        battery.showBatteryInfo();
    }
}
```

```
Модель компьютера: Lenovo
Заряд батареи: 80%
Ёмкость батареи: 5000 мАч
```

Локальный вложенный класс

Локальный класс — это класс, который объявляется внутри метода, конструктора или блока кода. Он существует и доступен только внутри этого метода или блока.

```
public class University {

    public void createStudent() {

        // Локальный вложенный класс
        class Student {

            String name;
            int age;

            // конструктор Student(String name, int age)

            void showInfo() {
                System.out.println("Студент: " + name +  ", возраст: " + age);
            }
        }

        // Используем локальный класс
        Student student = new Student("Анна", 20);

        student.showInfo();
    }
}
```

```
public static void main(String[] args) {

        University university = new University();

        university.createStudent();
    }
```

Анонимные классы

Анонимный класс — это класс без имени, который создаётся непосредственно в момент создания объекта. Его используют, когда нужно один раз создать объект с определённым поведением и нет необходимости создавать отдельный именованный класс.

```
abstract class Animal {

    abstract void makeSound();
}
```

```
Animal animal = new Animal() {

    @Override
    void makeSound() {
        System.out.println("Гав!");
    }
};

animal.makeSound();
```

Задание

Создай класс Game, который представляет компьютерную игру.
У него должны быть поля:

- title — название игры
- playerName — имя игрока
- score — количество очков

Внутри класса Game реализуй три вида вложенных классов.

1. Статический вложенный класс GameInfo
Хранит:

- жанр игры
- разработчика

Метод showInfo() выводит:
Игра: Minecraft
Жанр: Песочница
Разработчик: Mojang

2. Нестатический вложенный класс Player
Хранит

- уровень игрока

Метод showPlayerInfo() должен вывести:
Игрок: Алексей
Уровень: 15
Очки: 2500
При этом класс Player должен иметь доступ к полям внешнего класса Game.

3. Локальный класс Reward
Внутри метода giveReward() создай локальный класс Reward.
Он должен хранить название награды и количество бонусных очков.
Метод showReward() выводит:
Награда: Золотой кубок
Бонус: 500 очков

В main() должно быть примерно так:

```
Game game = new Game("Minecraft", "Алексей", 2500);

// Статический вложенный класс
Game.GameInfo info = new Game.GameInfo("Песочница", "Mojang");
info.showInfo();

// Нестатический вложенный класс
Game.Player player = game.new Player(15);
player.showPlayerInfo();

// Локальный класс
game.giveReward();
```